Correcting the ACE Request Scheduler and Measuring Communication-Memory Congestion in a Multi-Core Quantum Mesh

BTP, MARS Lab, IIT Guwahati

Draft of 24 October 2026

Contents

1 Scope and method **3**
- 1.1 What this report is about . . . 3
- 1.2 Ground rules followed . . . 3
- 1.3 How the work was organised . . . 4

2 Background **5**
- 2.1 The pipeline . . . 5
- 2.2 ACE and the native SeQUeNCe backend . . . 5
- 2.3 Policies . . . 5
- 2.4 Parameters in force . . . 5

3 What was wrong in the legacy simulator **6**
- 3.1 Reservation aliasing (F2) . . . 6
- 3.2 Event order is undefined (F9) . . . 7
- 3.3 Livelock under polling (F10) . . . 7
- 3.4 What the audit did not cover . . . 7

4 The corrections and how they were verified **8**
- 4.1 Making requests distinct (Gate 2) . . . 8
- 4.2 The per-qubit request manager (Gate 3) . . . 8
- 4.3 The checker . . . 8
- 4.4 Verification . . . 9
- 4.5 One latency definition for both schedulers . . . 9
- 4.6 The budget study rerun (Gate 6) . . . 9

5 Where the congestion is: heatmap and endpoint-versus-relay split **10**
- 5.1 Per-core congestion . . . 10
- 5.2 Own demand against relay demand (Experiment 2) . . . 11

6 Latency, fidelity and congestion across memory budgets (Experiment 1) **14**
- 6.1 Design . . . 14
- 6.2 Controls . . . 14
- 6.3 What congestion costs on-demand requests . . . 15
- 6.4 Congestion as waiting . . . 16
- 6.5 Pre-generation against on-demand, and the fidelity price . . . 17
- 6.6 Sensitivity to the physical parameters . . . 18

7 The memory cut-off time **19**
- 7.1 The timers . . . 19
- 7.2 What a discard does to compiler pairs . . . 19
- 7.3 What a discard does to the autonomous caches . . . 20
- 7.4 Decisions . . . 20

8 Can offline routing reduce the congestion? 21
8.1 Method . . . 21
8.2 Structural result . . . 22
8.3 Simulated effect . . . 22
8.4 The torus . . . 23
8.5 What can and cannot be concluded . . . 23

9 The torus topology 25
9.1 What changes . . . 25
9.2 Results . . . 25

10 Run times 27

11 Limits and open decisions 28

A Parameters in force 29
A.1 Where the physical values come from, and the open decision . . . 30

B Reproducing the results 31

Chapter 1

Scope and method

1.1 What this report is about

A multi-core quantum computer connects small processor cores over a network-on-chip. When a program moves a logical qubit from one core to another, the move is carried out by teleportation, which consumes an entangled pair (an EPR pair) shared between the two cores. Pre-generating those pairs before the program needs them can hide their generation latency, but every core has only a few *communication memories* in which pairs can wait, and many moves may need the same core at once. This report studies that contention (*congestion*) on a 4×4 mesh of cores.

The simulator used is *ACE*, an adaptive-continuous-entanglement simulator built on SeQUeNCe v0.8.1. The first part of the report documents what was found wrong with how ACE released requests to the network, how it was corrected, and how the correction was verified. The second part reports the experiments that the corrected simulator makes possible: latency, fidelity and waiting of ten benchmarks across communication-memory budgets (Experiment 1), the split of each core's congestion into its own endpoint demand and relay traffic (Experiment 2), a study of ACE's memory cut-off mechanisms, a study of whether offline routing can reduce congestion, and the same questions on a torus.

1.2 Ground rules followed

The following rules were fixed at the start and held throughout.

- Milliseconds measured in ACE are never compared with milliseconds from the native SeQUeNCe backend.
- Results from the legacy and the corrected scheduler are never merged in one table.
- A fallback to on-demand generation is not a failure, but a dropped request is.
- Nothing is claimed as measured unless it was run; mechanisms that were inferred are labelled as such.
- Congestion-aware routing is not claimed to exist: no routing policy is simulated, only bounded structurally (Chapter 8).
- Structural (trace-based) results are not called physical results.
- Physical parameters were not changed silently; the choice of parameters is an open decision (Appendix A).

1.3 How the work was organised

The work followed a gated plan, each gate committed separately on the branch `codex/ace-scheduler-correctness`:

Gate 0 a bit-for-bit baseline of the legacy simulator (164 result rows) that must still reproduce after every change;

Gate 1 an execution event log, an invariant checker, microcases and an audit of the legacy simulator;

Gate 2 making two requests that look alike distinct bookings (reservation identity);

Gate 3 replacing per-core serialization by a per-qubit request manager with event-driven admission;

Gate 4 verification on all ten benchmarks, three strategies, three mapping seeds and several memory budgets;

Gate 5 tracing every physical parameter to its source;

Gate 6 rerunning the earlier 378-run budget study under the corrected scheduler.

Chapter 2

Background

2.1 The pipeline

A benchmark circuit from MQT Bench is transpiled to the gate set $\{R_z, \sqrt{X}, X, \mathrm{CX}\}$ and cut into layers by an as-soon-as-possible schedule of its dependency graph. A qubit mapper (HQA, Hungarian-assignment based) places the logical qubits on cores layer by layer. Whenever the placement changes between two layers, the qubit has to move, and the move is expanded into one *hop* per link along a path found by Dijkstra's algorithm on the core graph: shortest in hops, with the product of link reliabilities as the tie-break. The result is a *trace*: for every layer, a list of hops (qubit, source core, destination core). Every hop of the traces used here joins two adjacent cores. The traces carry no gate-level dependencies, so a layer boundary stands in for them.

ACE then executes the trace: each hop is a request for one EPR pair between two adjacent cores, submitted through SeQUeNCe's resource-reservation protocol (RSVP), which books one communication memory at each end for a time window.

2.2 ACE and the native SeQUeNCe backend

ACE adds, on top of SeQUeNCe, adaptive continuous pre-generation of entanglement (cached pairs in spare memories), a compiler controller that pre-generates exactly the pairs the trace will need (the policies *fixed* and *dynamic*), and a request manager that submits a layer's requests together. The native backend instead uses a central scheduler with one generation job per link. The two backends are never compared in this report.

2.3 Policies

On-demand generates a pair only when a request asks for it, so it isolates contention. *Dynamic* compiler pre-generation prepares pairs ahead of the layer that needs them, using at most `budget - 1` memories per core as a compiler bank and leaving one for demand. Autonomous caches (*continuous*, *adaptive-continuous*) guess which pairs will be needed.

2.4 Parameters in force

Unless stated, the physical parameters are ACE's configuration: memory fidelity 0.95, memory efficiency 0.95, coherence time 1 s, detector efficiency 0.95, gate and measurement fidelity 0.99, classical delay 100 μs plus 20 μs per hop, and a router-to-BSM fibre of 0.025 (read as metres) with attenuation 0.2 dB/km. These come from the metro-scale adaptive-continuous protocol paper and differ from the group's multi-core paper in two values that matter (Appendix A).

Chapter 3

What was wrong in the legacy simulator

The audit ran the legacy simulator on the real SeQUeNCe stack with an execution-event recorder and an invariant checker attached (Chapter 4). Each finding below states its evidence: *source* (code read), *measured* (run), or both.

	Question	Answer
F1	Does ACE let two requests share a core?	No. `serialize_core_conflicts` splits every layer into batches with at most one request per core, whenever a compiler controller is attached. On QFT the peak demand per core falls from 24 to 1, so the memory budget cannot bind. Source and measured.
F2	Can two requests be mistaken for one?	**Yes.** SeQUeNCe's `Reservation` equality compares initiator, responder, window, size and fidelity, not the request identity. Serialization hid this in every legacy run. Source and measured.
F3	Does a rejection release earlier bookings?	Yes, exactly once on every node it travelled back through.
F4	Does the physics layer limit a link to one pair at a time?	**No.** 1 to 16 simultaneous requests on one link completed with zero retries.
F5	Phantom occupancy from compiler preparations?	Not in the 24 legacy audits: every booking was released.
F6	Is a trace layer a real barrier?	A conservative one; it is kept.
F7	A request that can never be admitted?	It retries for ever (found with a mistaken test case).
F8	Is waiting measured faithfully?	No: admission was polled every 1 ms.
F9	Resubmission at the instant a booking expires?	With short windows, stale or double bookings appear in 4 of 6 seeds; a delay of 1 ps removes them.
F10	Polling admission with one memory per core?	Two requests that swap two cores livelock: 25 000 submissions each.

Table 3.1: Findings of the audit. Full evidence is in `docs/reports/ace-correctness-audit.md`.

3.1 Reservation aliasing (F2)

With four requests on one link and 32 memories per core, the legacy code gave latencies of 1.2/55.7/62.2/67.9 ms with three retries and two orphaned pairs; the same requests, once they are distinct bookings, take 2.4/0.4/1.6/1.2 ms with no retry. The mechanism is that releasing a booking removes the *first equal* reservation, rules are matched to *any equal* reservation, and the application's dictionaries are keyed by reservation. Reverting only the equality reproduces the old behaviour and the checker flags it, which shows that the fix is the cause.

3.2 Event order is undefined (F9)

SeQUeNCe's event list is a binary heap ordered by time and priority only, so events at the same instant have no defined order. The legacy manager started a retry at the very instant the old booking expired. In an A/B test the only change was to resubmit 1 ps later: the violations disappeared. No code in the corrected scheduler relies on one same-instant event preceding another.

3.3 Livelock under polling (F10)

Randomized small traces found a request set that never completes under polling admission: requests 1 (core 8 to 12) and 3 (12 to 8), with one memory per core, each book their own memory, find the other's taken, roll back and retry on the same 1 ms period. The invariants held (nothing was corrupted), so this is a livelock, not corruption.

3.4 What the audit did not cover

One mapping seed and one physics seed for the 24 legacy audits; the autonomous strategies; and expiry of unused cached pairs, which was found later to never occur in the default runs and was then forced (Section **??**).

Chapter 4

The corrections and how they were verified

4.1 Making requests distinct (Gate 2)

Application reservations became a subclass whose equality and hash include the request identity. The protocol's `push` builds that class; adaptive (autonomous) reservations keep the stock contract. Tests cover equality, hashing, symmetry, timecard removal and the `push` path; a checker invariant (`ALIASED_RESERVATIONS`) detects a regression.

4.2 The per-qubit request manager (Gate 3)

The legacy manager is untouched and remains the default. The new manager, selected with `scheduler="per-qubit"`, applies these rules.

- **Chains.** The hops of one qubit within one layer form a chain in trace order; a hop is released when the previous hop has completed.
- **Independence.** Hops of different qubits are released together and compete for memory.
- **Layer barrier.** Layer $L+1$ starts when every request of layer L has completed; the 5 ms empty-layer gap of the legacy manager is kept.
- **Event-driven admission.** A request is handed to the reservation protocol only when both of its cores have a memory free for its whole window (read from the reservation tables, less requests admitted but not yet booked); the oldest request that fits goes first, and a release at any core wakes the line one picosecond later. A request that cannot start yet keeps the memory it would need (the *starvation guard*).
- **Timeouts** resubmit one picosecond after the window ends, one request at a time, not as a serialized retry layer.

No request waits while holding a memory that another needs, so a deadlock among application requests cannot form. The scheduler is modelled as a central controller that reads each core's reservation table without delay; that is an assumption about the system.

4.3 The checker

The invariant checker works on a typed event log (submissions, bookings, releases, pair deliveries and credits, completions). It checks, among others: no memory double-booked over overlapping windows; no release of a booking that was not live; no orphan pair; no credit beyond the request;

exactly one completion per request; no resubmission with a stale booking; a hop submitted only after its predecessor completed; the layer barrier. It was validated by deliberately breaking each rule, and it was corrected twice during the work: it first treated every booking as occupying a memory from the moment it was made (wrong for a booking of a later window), and it labelled reservations by memory address, which can be reused. Both faults were in the checker, not in ACE.

4.4 Verification

Check	Result
Legacy baseline (164 rows, 22 cases), after every change	reproduced exactly, under several Python hash seeds
Ten workloads × three strategies × 2 and 4 memories × mapping seeds 42, 43, 44 (360 audited cells)	every request completes, no invariant violated
GHZ against the legacy manager	identical event logs (on-demand, dynamic)
Random small traces at 1, 2, 4, 16 memories	all complete, never more bookings than memories
Forced expiry of cached pairs, both schedulers	all complete, no violation
Python hash seeds 1 and 2	identical makespan, events and retries

Table 4.1: Verification summary (`docs/reports/ace-scheduler-verification.md`).

4.5 One latency definition for both schedulers

The legacy manager's per-request latency starts at a request's first submission, so time spent waiting for earlier per-core batches is not counted, while the new manager counts admission waiting. They are not comparable as defined. Throughout this report latency means *common latency*: from the moment a request could first be asked for (its layer starting, or the previous hop of its qubit completing) to its completion, minus the fixed 5.3 ms pre-generation lead.

4.6 The budget study rerun (Gate 6)

The earlier 378-run budget study could not measure congestion because of the serialization. Rerun with the legacy scheduler through a parameterized driver, it reproduces the published results in every column (0 differing cells of 378×58), so the driver is faithful. Under the corrected scheduler the on-demand latency now depends on the budget: for QFT it falls from 4.72 ms at 2 memories to 1.13 ms at 32 (Chapter 6).

Chapter 5

Where the congestion is: heatmap and endpoint-versus-relay split

5.1 Per-core congestion

For every trace and layer, the *demand* of a core is the number of hops that touch it (every hop joins two adjacent cores, so a hop counts at both ends). A core is *overloaded* in a layer when its demand exceeds the number of communication slots, here 4. The percentage is over *communication-active layers* (layers with at least one hop), averaged with equal weight over the three mapping seeds 42, 43 and 44 of the same circuit. This is a structural count of the mapped traces: it simulates no physics.

The figure was regenerated from the raw traces without reading the archive's precomputed congestion files, as an independent check. All 160 cells (ten workloads, sixteen cores) match the project's earlier figure at its printed resolution, and a regression test pins every cell.

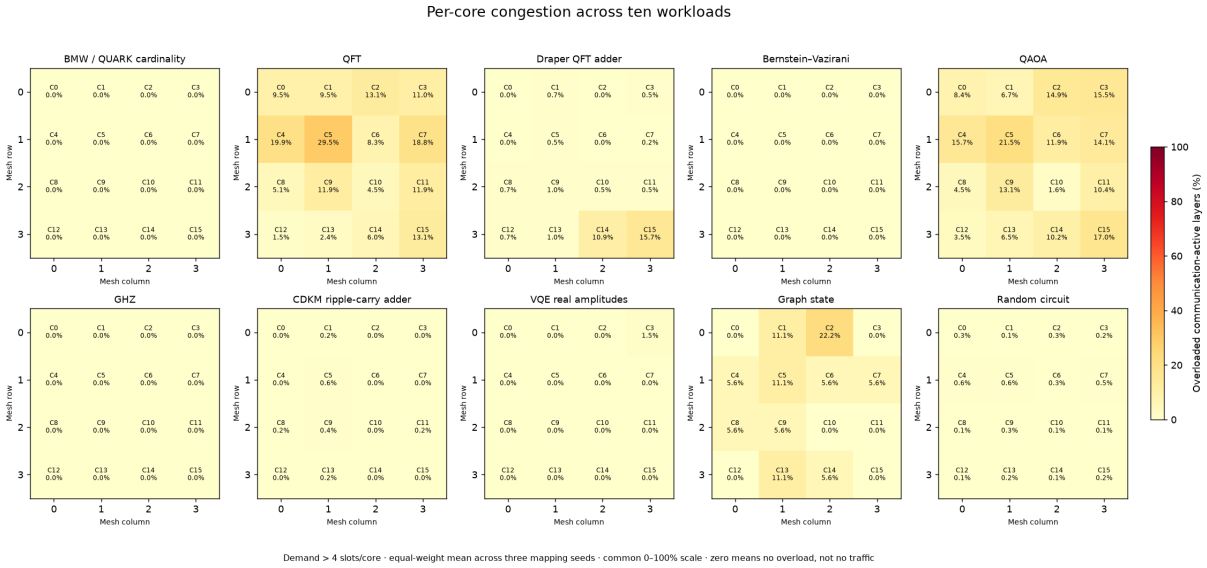


Figure 5.1: Per-core congestion: percentage of communication-active layers in which a core's demand exceeds 4 slots (mean over three mapping seeds). All 160 cells match the project's earlier figure at 0.1% resolution.

Workload	Active layers (seeds 42/43/44)	Layers with any overload	Hottest core
GHZ	15/15/15	0.0%	C0 (0.0%)
Bernstein–Vazirani	16/16/16	0.0%	C0 (0.0%)
BMW/QUARK cardinality	90/90/90	0.0%	C0 (0.0%)
VQE real amplitudes	22/22/22	6.1%	C9 (4.5%)
CDKM ripple-carry adder	157/157/157	0.6%	C6 (0.6%)
Graph state	6/6/6	33.3%	C5 (33.3%)
Draper QFT adder	138/138/138	41.3%	C10 (23.4%)
QAOA	163/163/163	71.6%	C10 (56.0%)
QFT	112/112/112	65.2%	C10 (58.0%)
Random circuit	2296/2296/2296	8.0%	C9 (3.2%)

Table 5.1: Structural overload at capacity 4, pooled over the three mapping seeds.

Congestion concentrates on the central cores, which lie on the most shortest paths. QFT and QAOA are the heavy cases, graph state is overloaded in a third of its (few) active layers, and GHZ, Bernstein–Vazirani and BMW never exceed four.

5.2 Own demand against relay demand (Experiment 2)

A *route* is a qubit's consecutive hops within one layer; its first source and final destination are its *endpoints*, every core in between is a *relay*. For a core and a layer, D counts the hops touching the core, D_{end} those where the core is an endpoint of the route, and D_{relay} the rest, so that a relay (touched by an arriving and a leaving hop) contributes two: $D = D_{\text{end}} + D_{\text{relay}}$. With capacity C, a layer is overloaded for the core when $D > C$. It is *own-overloaded* when $D_{\text{end}} > C$, and *relay-only-overloaded* when $D_{\text{end}} \leq C < D$. The two sets are disjoint and together are exactly the overloaded layers.

Validation. Own plus relay-only reproduces all 160 heatmap cells, and the per-core relay incidences, incident hops and peak layer demand reproduce all 480 per-trial rows of the archive's per-core file exactly.

Counting qubits in flight. A second variant counts routes instead of hops: under per-qubit ordering the hops of a route run one after another, so a relay needs one memory for the route at a time, not two.

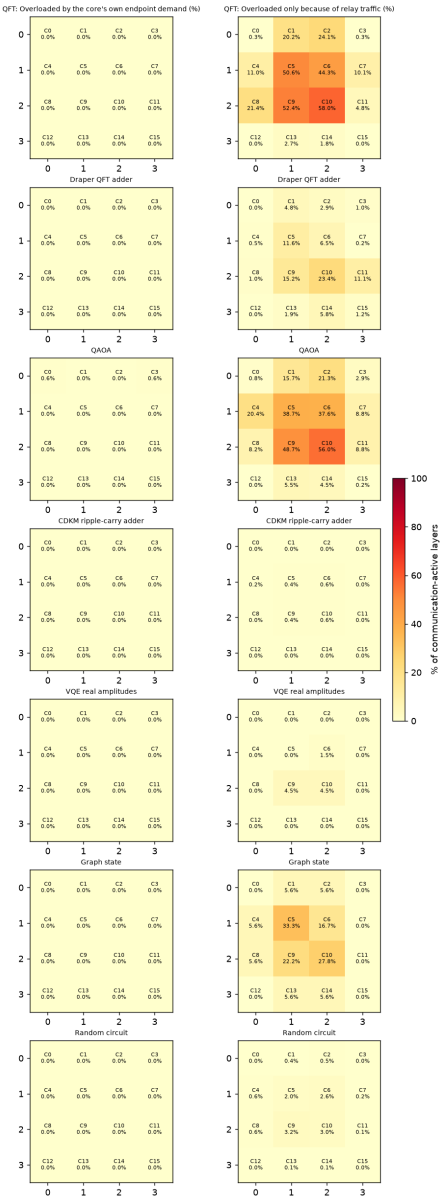


Figure 5.2: Per core, the share of layers overloaded by the core's own endpoint demand (left) and only because of relay traffic (right), at capacity 4, hop counting. Workloads without overload are omitted.

Workload	C=1	C=2	C=3	C=4
GHZ	0.0%	none	none	none
Bernstein–Vazirani	0.0%	none	none	none
BMW/QUARK cardinality	0.0%	none	none	none
VQE real amplitudes	2.0%	0.0%	0.0%	0.0%
CDKM ripple-carry adder	0.4%	0.0%	0.0%	0.0%
Graph state	26.0%	9.5%	0.0%	0.0%
Draper QFT adder	54.5%	6.9%	6.4%	0.0%
QAOA	51.9%	14.9%	2.3%	0.4%
QFT	76.8%	10.1%	7.1%	0.0%
Random circuit	15.7%	1.8%	0.0%	0.0%

Table 5.2: Share of overloaded core-layers caused by a core's own endpoint demand (hop counting); the rest is relay-only overload.

Workload	C=1	C=2	C=3	C=4
GHZ	none	none	none	none
Bernstein–Vazirani	none	none	none	none
BMW/QUARK cardinality	none	none	none	none
VQE real amplitudes	5.3%	0.0%	none	none
CDKM ripple-carry adder	10.7%	0.0%	0.0%	0.0%
Graph state	37.5%	18.2%	0.0%	0.0%
Draper QFT adder	76.0%	9.5%	17.1%	0.0%
QAOA	59.7%	20.7%	4.5%	1.2%
QFT	86.8%	11.5%	12.8%	0.0%
Random circuit	39.6%	7.0%	0.0%	0.0%

Table 5.3: Share of overloaded core-layers caused by a core's own endpoint demand (route counting); the rest is relay-only overload.

Findings. At the modelled capacity of four, almost all overload is relay overload. The peak endpoint demand is 4 for QFT and Draper, 5 for QAOA and at most 3 for the others, against a total demand up to 24. The only own-overload at capacity 4 is in QAOA, at the corner cores C0 and C3 (0.61% of active layers each), which is 0.44% of QAOA's overload. Smaller budgets bring own demand in: QFT is 77% own-overload at capacity 1. A routing change can therefore remove at most the relay part; the own part is a floor set by the mapping. Route counting finds less overload than hop counting (core C10 of QFT is overloaded in 46.4% of active layers against 58.0%) and relay traffic is still essentially all of what remains.

Chapter 6

Latency, fidelity and congestion across memory budgets (Experiment 1)

6.1 Design

All ten benchmarks, on the corrected scheduler, with $1, 2, 3, 4, 6, 8, 16, 32$ communication memories per core (QFT also 64). Budgets 1–4 are *modelled* (the group's multi-core paper gives four communication qubits per core); 6–32 are *counterfactual* (what-if capacity), marked *. Two policies: *on-demand* and *dynamic* compiler pre-generation with budget − 1 memories as compiler bank and one held for demand (so dynamic starts at two). Three mapping seeds and three physics seeds give nine runs per cell, 1 368 runs in total. Latency is the common latency defined in Chapter 4.

6.2 Controls

- PASS every run completes every request — 1368 rows, 0 not ok
- PASS zero-congestion control: GHZ on-demand latency does not respond to the budget — 0 of 9 (mapping, physics) runs vary across budgets
- PASS saturation: QFT with 64 memories reproduces 32 run for run (relative 1e-6) — 18 runs compared, largest relative difference 7.96e-12
- PASS on-demand delivered fidelity does not depend on the budget — 0 of 90 runs vary
- PASS on-demand latency does not rise when memory is added, beyond seed-to-seed noise — 639 budget steps: 1 rise by less than the physics-seed spread (largest 0.0005 ms), 0 rise by more

The first full run failed two controls for reasons that were not physical: the saturation check compared to exact equality although a few requests "waited" the one picosecond the scheduler leaves after a release, and one on-demand step rose by 0.0005 ms with more memory (a thirtieth of the seed-to-seed spread), which is sampling noise because a delayed request draws different random numbers. The controls were refined, tests pin them, and a request now "waited" only if it waited more than a microsecond. The rerun is numerically identical to the first run in every column except that redefined one, which also shows that the pipeline is deterministic.

6.3 What congestion costs on-demand requests

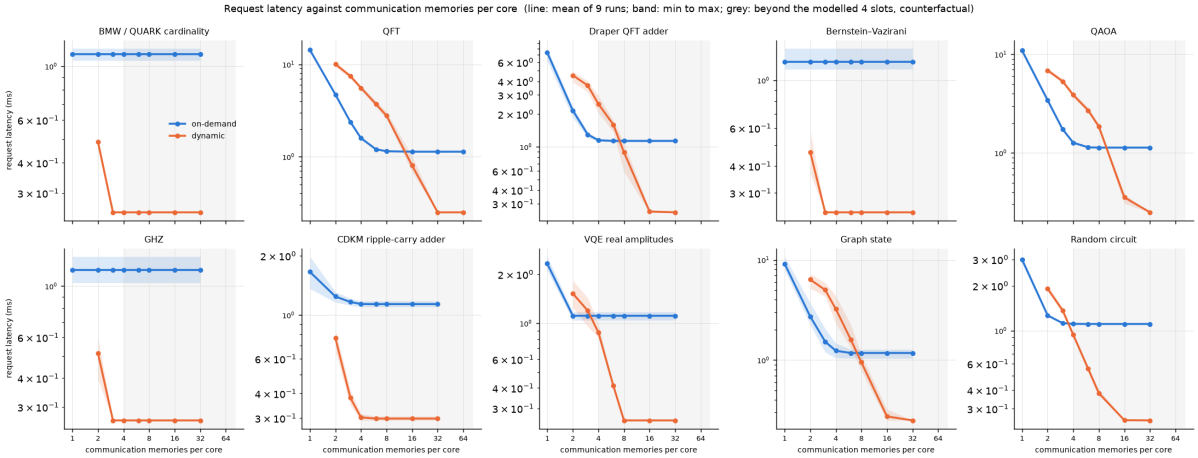


Figure 6.1: Request latency against communication memories per core. Line: mean of nine runs; band: minimum to maximum; grey: counterfactual budgets.

Workload	1	2	3	4	6*	8*	16*	32*
GHZ	1.17±0.11	1.17±0.11	1.17±0.11	1.17±0.11	1.17±0.11	1.17±0.11	1.17±0.11	1.17±0.11
Bernstein–Vazirani	1.22±0.12	1.22±0.12	1.22±0.12	1.22±0.12	1.22±0.12	1.22±0.12	1.22±0.12	1.22±0.12
BMW/QUARK cardinality	1.12±0.04	1.12±0.04	1.12±0.04	1.12±0.04	1.12±0.04	1.12±0.04	1.12±0.04	1.12±0.04
VQE real amplitudes	2.32±0.14	1.11±0.05	1.11±0.05	1.11±0.05	1.11±0.05	1.11±0.05	1.11±0.05	1.11±0.05
CDKM ripple-carry adder	1.66±0.22	1.24±0.05	1.17±0.03	1.14±0.03	1.14±0.03	1.14±0.03	1.14±0.03	1.14±0.03
Graph state	9.14±1.14	2.72±0.54	1.53±0.30	1.24±0.13	1.18±0.08	1.18±0.08	1.18±0.08	1.18±0.08
Draper QFT adder	7.36±0.84	2.16±0.20	1.30±0.05	1.15±0.02	1.13±0.02	1.13±0.02	1.13±0.02	1.13±0.02
QAOA	10.92±0.41	3.42±0.14	1.74±0.06	1.27±0.02	1.14±0.01	1.13±0.01	1.13±0.01	1.13±0.01
QFT	14.42±0.35	4.72±0.14	2.39±0.09	1.59±0.05	1.21±0.02	1.15±0.02	1.13±0.02	1.13±0.02
Random circuit	2.99±0.10	1.27±0.02	1.12±0.01	1.11±0.01	1.11±0.01	1.11±0.01	1.11±0.01	1.11±0.01

Table 6.1: Request latency (ms), on-demand, mean ± sd over nine runs. Budgets marked * are counterfactual (beyond the modelled four slots).

Three workloads with no overload (GHZ, Bernstein–Vazirani, BMW) do not respond to the budget at all. At one memory the congested workloads are severe: QFT is about 12.7 times its saturated latency, QAOA 9.6, graph state 7.8 and Draper 6.5. At the modelled four memories the penalty against 32 memories is +40% for QFT, +12% for QAOA, +5% for graph state and +1% for Draper; the other workloads are within a percent. The floor, reached once nothing waits, is 1.11 to 1.22 ms in every workload: that is generation time, not contention.

6.4 Congestion as waiting

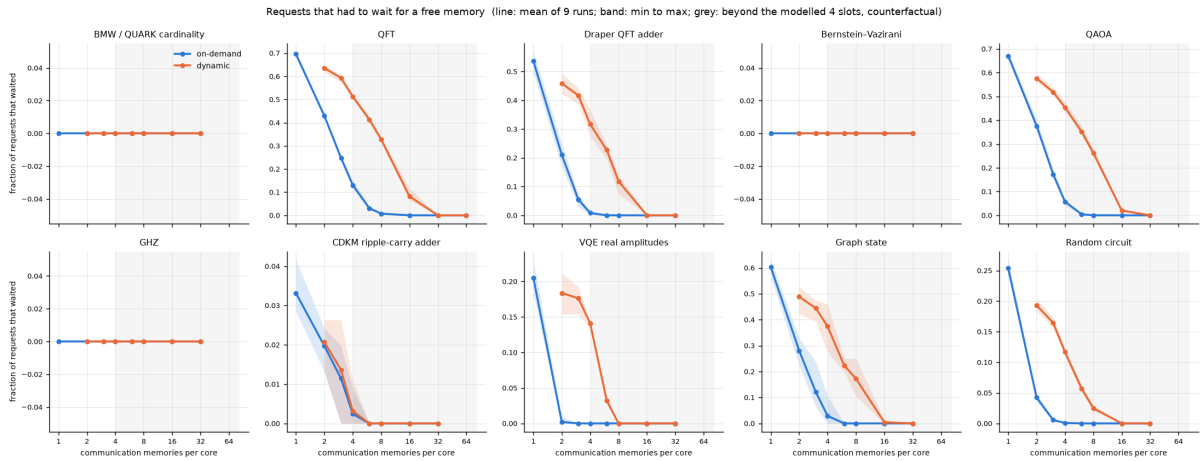


Figure 6.2: Fraction of requests that had to wait for a free memory. Line: mean of nine runs; band: minimum to maximum; grey: counterfactual budgets.

Workload	1	2	3	4	6*	8*	16*	32*
GHZ	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
Bernstein–Vazirani	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
BMW/QUARK cardinality	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
VQE real amplitudes	0.205	0.002	0.000	0.000	0.000	0.000	0.000	0.000
CDKM ripple-carry adder	0.033	0.020	0.011	0.002	0.000	0.000	0.000	0.000
Graph state	0.602	0.281	0.121	0.029	0.000	0.000	0.000	0.000
Draper QFT adder	0.536	0.211	0.055	0.008	0.000	0.000	0.000	0.000
QAOA	0.671	0.376	0.172	0.058	0.004	0.000	0.000	0.000
QFT	0.696	0.430	0.249	0.130	0.031	0.007	0.000	0.000
Random circuit	0.254	0.042	0.006	0.001	0.000	0.000	0.000	0.000

Table 6.2: Fraction of requests that waited more than a microsecond for a free memory, on-demand. Budgets marked * are counterfactual (beyond the modelled four slots).

The structural overload of the traces at the same capacity (Chapter 5) predicts which workloads pay: across the 60 (workload, budget 1 to 8) points the structural overload and the on-demand latency penalty are strongly rank-correlated (Spearman $\rho = 0.85$).

6.5 Pre-generation against on-demand, and the fidelity price

Workload	2	3	4	6*	8*	16*	32*
GHZ	0.51±0.07	0.26±0.00	0.26±0.00	0.26±0.00	0.26±0.00	0.26±0.00	0.26±0.00
Bernstein–Vazirani	0.46±0.07	0.24±0.00	0.24±0.00	0.24±0.00	0.24±0.00	0.24±0.00	0.24±0.00
BMW/QUARK cardinality	0.49±0.02	0.25±0.00	0.25±0.00	0.25±0.00	0.25±0.00	0.25±0.00	0.25±0.00
VQE real amplitudes	1.52±0.20	1.19±0.16	0.88±0.07	0.41±0.00	0.25±0.00	0.25±0.00	0.25±0.00
CDKM ripple-carry adder	0.76±0.06	0.38±0.03	0.30±0.01	0.30±0.01	0.30±0.01	0.30±0.01	0.30±0.01
Graph state	6.36±0.79	5.01±0.59	3.23±0.82	1.60±0.39	0.95±0.13	0.27±0.04	0.25±0.00
Draper QFT adder	4.55±0.43	3.67±0.34	2.48±0.38	1.60±0.17	0.89±0.23	0.25±0.01	0.25±0.00
QAOA	6.89±0.28	5.33±0.19	3.88±0.17	2.70±0.13	1.86±0.08	0.35±0.04	0.25±0.00
QFT	10.06±0.54	7.49±0.33	5.57±0.21	3.73±0.25	2.78±0.19	0.80±0.16	0.25±0.00
Random circuit	1.91±0.08	1.37±0.04	0.94±0.02	0.56±0.02	0.38±0.02	0.25±0.00	0.25±0.00

Table 6.3: Request latency (ms), dynamic pre-generation. Budgets marked * are counterfactual (beyond the modelled four slots).

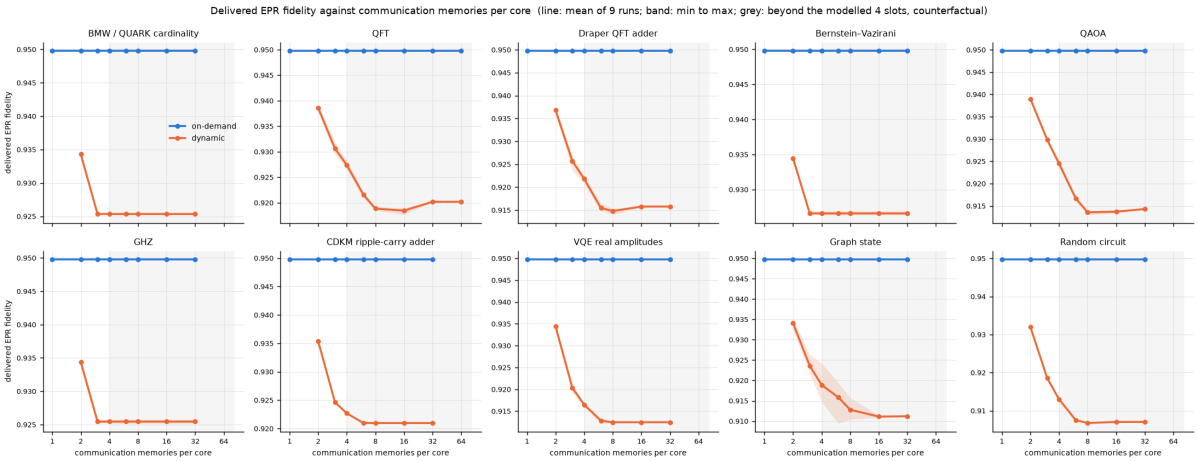


Figure 6.3: Delivered EPR fidelity against communication memories per core. Line: mean of nine runs; band: minimum to maximum; grey: counterfactual budgets.

Workload	2	3	4	6*	8*	16*	32*
GHZ	0.9344	0.9254	0.9254	0.9254	0.9254	0.9254	0.9254
Bernstein–Vazirani	0.9345	0.9266	0.9266	0.9266	0.9266	0.9266	0.9266
BMW/QUARK cardinality	0.9343	0.9253	0.9253	0.9253	0.9253	0.9253	0.9253
VQE real amplitudes	0.9344	0.9203	0.9165	0.9128	0.9125	0.9125	0.9125
CDKM ripple-carry adder	0.9354	0.9247	0.9227	0.9210	0.9210	0.9210	0.9210
Graph state	0.9341	0.9236	0.9189	0.9158	0.9128	0.9111	0.9112
Draper QFT adder	0.9368	0.9257	0.9218	0.9154	0.9148	0.9158	0.9158
QAOA	0.9390	0.9298	0.9245	0.9167	0.9136	0.9137	0.9143
QFT	0.9386	0.9306	0.9274	0.9215	0.9189	0.9185	0.9202
Random circuit	0.9320	0.9186	0.9130	0.9075	0.9068	0.9070	0.9071

Table 6.4: Delivered EPR fidelity, dynamic pre-generation. Budgets marked * are counterfactual (beyond the modelled four slots).

For the congested workloads dynamic is slower than on-demand at small budgets, because its compiler bank takes budget − 1 of the memories and leaves one for demand; the first budget at which dynamic is faster than on-demand is 8 for graph state and Draper and 16 for QAOA and QFT. At 32 memories dynamic reaches 0.24 to 0.30 ms against 1.11 to 1.22 ms on-demand.

Delivered fidelity under dynamic is below the on-demand value (0.9498 in every cell) and falls as the compiler bank grows from two to about eight memories, by at most 0.043 under ACE's one-second coherence time; the mechanism (pairs stored longer) is likely but was not measured.

6.6 Sensitivity to the physical parameters

This is exploratory, not a result of the study: the parameters are undecided. Five workloads, budgets $2, 4, 8, 32$, mapping seed 42, were rerun with the group's multi-core paper's coherence time (1 ms) and memory efficiency (0.85) in place of ACE's (1 s and 0.95). With the paper's values **every cached pair expires unused** (6 of 15 700 compiler pairs were used), so dynamic pre-generation gives no benefit at all; on-demand fidelity falls from 0.9498 to 0.7861 in every workload and budget, and on-demand latency rises by about 0.26 to 0.42 ms. The congestion curves keep their shape. So the congestion results are structural and change little with these parameters, while the pre-generation and fidelity results depend entirely on them.

Chapter 7

The memory cut-off time

A *memory cut-off time* is a threshold after which a stored EPR pair is discarded and a new pair generated. Nothing new needs to be built to have one: ACE already has it, as the SeQUeNCe memory's `cutoff_ratio`, and two other timers also end a stored pair's life.

7.1 The timers

Memory cut-off. When a pair is stored, an expiry is scheduled `cutoff_ratio` × coherence time later. At expiry the memory is reset (unless it is in an application) and returned to the free state; ACE then drops the pair from its cache and tells the compiler controller it expired. ACE's configuration never sets the ratio, so it is 1 and the cut-off equals the coherence time, 1 s. A ratio written in the topology template was verified to reach every memory.

Reservation window. A pair's booking ends with its window; an unused pair is released then. The window is 1000 ms for compiler pairs and, for the autonomous caches, runs to the next 0.2 s period boundary.

Planner lifetime. The planner's `-coherence-time-layers` (10) says how far ahead a pair may be prepared; it is a planning assumption, tied to nothing physical.

Separately the stored state decoheres continuously at the rate 1/coherence time, lowering fidelity but discarding nothing.

7.2 What a discard does to compiler pairs

Workload	Case	Done	Viol.	Consumed	Window	Cut-off	Bookings	Makespan ms
ghz	ACE default: cut-off = coherence time (1 s), window 1000 ms	24/24	0	48	0	0	48	303.5
ghz	short reservation window only (20 ms)	24/24	0	36	12	0	48	311.2
ghz	cut-off ratio 0.02 (20 ms), window 1000 ms	24/24	0	34	12	12	55	311.4
ghz	cut-off ratio 0.005 (5 ms), window 1000 ms	24/24	0	0	34	34	49	322.0
ghz	cut-off ratio 0.02 and window 20 ms	24/24	0	36	12	0	48	311.2
graphstate	ACE default: cut-off = coherence time (1 s), window 1000 ms	76/76	0	104	0	0	134	274.5
graphstate	short reservation window only (20 ms)	76/76	0	48	56	0	134	241.3
graphstate	cut-off ratio 0.02 (20 ms), window 1000 ms	76/76	0	24	32	32	179	351.8
graphstate	cut-off ratio 0.005 (5 ms), window 1000 ms	76/76	0	0	34	34	339	347.4
graphstate	cut-off ratio 0.02 and window 20 ms	76/76	0	48	56	0	134	242.5

Table 7.1: Which timer discards a cached pair (compiler pre-generation, 4 memories). Consumed: pairs used; Window: releases at the end of the reservation window; Cut-off: memory resets by the cut-off.

The earlier timer wins: with both at 20 ms the window fires first and no cut-off reset occurs. For compiler pairs a cut-off discards the pair and nothing replaces it: the controller only marks

it expired, the request that wanted it generates its own pair on demand, and all requests still complete. The run gets longer (graph state: 274.5 ms makespan by default, 351.8 ms with a 20 ms cut-off, 347.4 ms with 5 ms). In every case the number of window releases equals the number of cut-off resets, consistent with a discarded pair's booking staying until its window ends (inferred from the counts).

7.3 What a discard does to the autonomous caches

Strategy	Cut-off ratio	Generated	Used	Reported waste	Real unused	Latency ms	Fidelity
continuous	1 (default)	44936	90	99.7%	99.80%	5.147	0.9297
adaptive-continuous	1 (default)	5184	120	97.8%	97.69%	4.882	0.9258
continuous	0.2	44936	90	99.7%	99.80%	5.147	0.9297
adaptive-continuous	0.2	5184	120	97.8%	97.69%	4.882	0.9258
continuous	0.02	447388	122	10.0%	99.97%	4.865	0.9354
adaptive-continuous	0.02	41472	122	12.0%	99.71%	4.865	0.9354

Table 7.2: Autonomous caches on Random-16 (4 memories, coherence 1 s): effect of the memory cut-off. Reported waste omits cut-off discards.

A cut-off of 200 ms changes nothing, because the cache's window already ends at the next 0.2 s boundary. At 20 ms the cache discards and *regenerates*: ten times more pairs are generated, the used pairs are fresher (fidelity 0.9297 to 0.9354) and latency falls slightly (5.15 to 4.87 ms). ACE's waste counter does not see cut-off discards (it is incremented only when a generation rule expires at the window end), so the reported waste drops from 99.7% to 10.0% although 99.97% of the generated pairs went unused; waste measured with a cut-off must be computed as $1 - \text{used/generated}$.

7.4 Decisions

Whether "cut-off" should mean discard-and-regenerate (the caches already do; compiler pre-generation falls back on demand and re-preparing would be a new feature); exposing the cut-off in milliseconds as a recorded study parameter; tying the planner's lifetime to it and fixing the waste counter first; and the coherence time, which comes before any cut-off: with 1 s coherence a cut-off never acts, and with the group's 1 ms a cut-off equal to the coherence time would discard every cached pair before use.

Chapter 8

Can offline routing reduce the congestion?

Experiment 2 (Chapter 5) showed that at the modelled capacity of four memories nearly all overload is relay overload: it is caused by routes passing through the central cores, not by routes starting or ending there. That raises a question about the routes. The mapper chooses, for every qubit move, one Dijkstra shortest path (fewest hops, link reliability as the tie-break), without looking at what the other moves of the layer are doing. Many alternative shortest paths exist: between opposite corners of the 4×4 mesh there are $\binom{6}{3} = 20$. Could choosing them *offline*, with the mapping untouched, spread the relay load and reduce the congestion?

8.1 Method

The mapping is held fixed: every qubit still moves from the same core to the same core in the same layer. Only the path of each move (a *route*) is re-chosen, layer by layer (layers are independent because a layer's routes start and end at fixed cores). For each layer that has any overload, the choice of one candidate path per route is made by solving a small mixed-integer program exactly, minimising the *excess*, the demand above capacity summed over cores. The candidates are all simple paths of at most the shortest length plus 0 or 2 hops, and the mapper's own path is always a candidate, so the result is never worse than the baseline.

Two lower bounds hold for *any* routing, so they show how much is left to gain. Endpoints do not depend on the path, so the demand of a core is at least its endpoint demand; and a route of shortest length k puts at least $2k$ hop touches ($k+1$ route touches) on the cores, so the total excess is at least the minimum total demand minus (cores $\times$ capacity). Both are tested.

Demand is counted two ways: by hops (the archive's counting) and by routes, the number of qubits in flight through a core, which is what the per-qubit scheduler experiences.

8.2 Structural result

Capacity	Extra hops	Excess now	Excess after re-routing	Lower bound	Overloaded layers	Added hops
1	0	27043	21894 (19.0%)	18336	4110 → 3801	0.00%
1	2	27043	21018 (22.3%)	18336	4110 → 3621	2.55%
2	0	10385	6771 (34.8%)	4995	1641 → 978	0.00%
2	2	10385	6288 (39.5%)	4995	1641 → 840	1.48%
3	0	4774	1791 (62.5%)	846	855 → 420	0.00%
3	2	4774	1497 (68.6%)	846	855 → 327	0.98%
4	0	2216	348 (84.3%)	78	482 → 102	0.00%
4	2	2216	237 (89.3%)	78	482 → 48	0.36%

Table 8.1: Offline re-routing, mesh, route counting, pooled over ten benchmarks and three mapping seeds. Excess is demand above capacity.

Pooled over the ten benchmarks, the best routing removes most of the excess at the modelled capacity: with route counting at capacity four, shortest paths alone cut the excess from 2 216 to 348 (a 84.3% reduction) and the layers with any overload from 482 to 102; allowing two extra hops cuts it to 237 (89.3%) at a cost of 0.36% more hops. The gain shrinks as the capacity shrinks (19.0% at capacity one), because there the total relay volume, not the choice of path, is the limit, which the volume bound shows. With hop counting the reductions are smaller (65.6% at capacity four).

8.3 Simulated effect

The structural gain would be an artefact if it did not survive the simulator, so the re-routed traces were run in ACE next to the original routes, on-demand, at the budget they were optimised for, with the same mapping, endpoints, physics seeds and parameters. Three mapping seeds and three physics seeds give nine paired runs per cell.

Workload	Mem.	Original	Re-routed	+2 hops	Waited
Graph state	2	2.72	1.53 (+42.3% ±9.9; 9/9)	1.41 (+46.6% ±8.9; 9/9)	0.281 → 0.086
Graph state	3	1.53	1.15 (+22.9% ±11.8; 9/9)	1.19 (+19.6% ±13.5; 9/9)	0.121 → 0.000
Graph state	4	1.24	1.16 (+6.0% ±7.6; 7/9)	1.21 (+1.9% ±8.9; 4/9)	0.029 → 0.000
Draper QFT adder	2	2.16	1.69 (+21.1% ±7.9; 9/9)	1.48 (+31.1% ±6.8; 9/9)	0.211 → 0.115
Draper QFT adder	3	1.30	1.16 (+10.3% ±3.3; 9/9)	1.16 (+10.5% ±3.7; 9/9)	0.055 → 0.007
Draper QFT adder	4	1.15	1.13 (+1.3% ±1.3; 8/9)	1.14 (+1.1% ±0.8; 9/9)	0.008 → 0.000
QAOA	2	3.42	2.24 (+34.5% ±2.4; 9/9)	2.12 (+38.0% ±2.4; 9/9)	0.376 → 0.206
QAOA	3	1.74	1.25 (+28.4% ±2.3; 9/9)	1.19 (+31.4% ±2.2; 9/9)	0.172 → 0.037
QAOA	4	1.27	1.13 (+10.9% ±1.7; 9/9)	1.13 (+11.5% ±1.6; 9/9)	0.058 → 0.003
QFT	2	4.72	3.49 (+26.1% ±2.4; 9/9)	3.03 (+35.9% ±2.1; 9/9)	0.430 → 0.312
QFT	3	2.39	1.53 (+35.9% ±2.2; 9/9)	1.47 (+38.2% ±2.3; 9/9)	0.249 → 0.097
QFT	4	1.59	1.19 (+25.2% ±2.4; 9/9)	1.17 (+26.6% ±2.0; 9/9)	0.130 → 0.018
Random circuit	2	1.27	1.13 (+10.6% ±1.3; 9/9)	1.12 (+11.7% ±1.3; 9/9)	0.042 → 0.007
Random circuit	3	1.12	1.11 (+1.2% ±0.3; 9/9)	1.11 (+1.3% ±0.3; 9/9)	0.006 → 0.000
Random circuit	4	1.11	1.11 (+0.2% ±0.1; 9/9)	1.11 (+0.2% ±0.2; 8/9)	0.001 → 0.000

Table 8.2: Simulated on-demand latency (ms), mesh: original routes against offline re-routing. Brackets: paired reduction, sd, pairs faster of nine.

Re-routing alone makes on-demand requests faster in nearly every pair. At two memories the latency falls by 26% to 42% on the congested workloads; at the modelled four memories by 25% for QFT, 11% for QAOA and 6% for graph state, and by 1% or less where there was little congestion. The fraction of requests that had to wait falls accordingly. Two extra hops help at

two memories (QFT 26% to 36%) but not at four, where the added hop count costs about as much as it saves.

8.4 The torus

On the torus, which has more paths between cores, the optimum is closer to the lower bound still: at capacity four the excess falls from 493 to 15 (97.0%), against a lower bound of 6, and with two extra hops it reaches the bound.

Capacity	Extra hops	Excess now	Excess after re-routing	Lower bound	Overloaded layers	Added hops
1	0	20202	15858 (21.5%)	14847	3826 → 3339	0.00%
1	2	20202	15387 (23.8%)	14847	3826 → 3312	1.73%
2	0	6108	3351 (45.1%)	2847	1319 → 702	0.00%
2	2	6108	3108 (49.1%)	2847	1319 → 630	0.93%
3	0	1852	363 (80.4%)	303	562 → 162	0.00%
3	2	1852	342 (81.5%)	303	562 → 159	0.09%
4	0	493	15 (97.0%)	6	220 → 15	0.00%
4	2	493	6 (98.8%)	6	220 → 6	0.04%

Table 8.3: Offline re-routing, torus, route counting, pooled over ten benchmarks and three mapping seeds. Excess is demand above capacity.

Simulated on the torus, re-routing helps at two memories (QFT −15%, QAOA −32%, Draper −36% latency) but little at the modelled four (QFT −6.8%, QAOA −3.2%, others about 2% or none), because the torus is already close to its latency floor there.

Workload	Mem.	Original	Re-routed	+2 hops	Waited
Graph state	2	1.86	1.27 (+31.2% ±7.2; 9/9)	1.28 (+31.0% ±6.9; 9/9)	0.157 → 0.033
Graph state	3	1.25	1.10 (+11.0% ±7.4; 8/9)	1.12 (+10.3% ±8.7; 8/9)	0.033 → 0.000
Graph state	4	1.11	1.12 (-0.4% ±1.7; 1/9)	1.11 (+0.3% ±3.9; 2/9)	0.000 → 0.000
Draper QFT adder	2	2.25	1.45 (+35.6% ±1.9; 9/9)	1.26 (+44.0% ±1.4; 9/9)	0.226 → 0.075
Draper QFT adder	3	1.28	1.15 (+10.5% ±1.8; 9/9)	1.14 (+10.7% ±1.0; 9/9)	0.053 → 0.007
Draper QFT adder	4	1.13	1.11 (+1.8% ±0.6; 9/9)	1.11 (+1.6% ±0.8; 9/9)	0.008 → 0.000
QAOA	2	2.47	1.66 (+32.3% ±4.7; 9/9)	1.62 (+34.2% ±4.7; 9/9)	0.265 → 0.111
QAOA	3	1.40	1.15 (+17.6% ±4.6; 9/9)	1.14 (+18.3% ±4.3; 9/9)	0.083 → 0.004
QAOA	4	1.17	1.13 (+3.2% ±2.8; 9/9)	1.13 (+3.1% ±2.6; 9/9)	0.016 → 0.000
QFT	2	2.79	2.37 (+14.7% ±7.3; 9/9)	2.42 (+12.7% ±7.8; 9/9)	0.303 → 0.221
QFT	3	1.51	1.21 (+19.3% ±6.0; 9/9)	1.24 (+17.3% ±6.1; 9/9)	0.102 → 0.023
QFT	4	1.21	1.13 (+6.8% ±3.2; 9/9)	1.12 (+7.3% ±3.0; 9/9)	0.028 → 0.001
Random circuit	2	1.19	1.11 (+6.1% ±0.9; 9/9)	1.11 (+6.3% ±0.9; 9/9)	0.022 → 0.002
Random circuit	3	1.11	1.11 (+0.4% ±0.1; 9/9)	1.11 (+0.3% ±0.1; 9/9)	0.001 → 0.000
Random circuit	4	1.11	1.11 (-0.0% ±0.1; 1/9)	1.11 (+0.0% ±0.1; 3/9)	0.000 → 0.000

Table 8.4: Simulated on-demand latency (ms), torus: original routes against offline re-routing. Brackets: paired reduction, sd, pairs faster of nine.

8.5 What can and cannot be concluded

Offline routing *can* reduce congestion: with the mapping fixed, an exact offline choice of paths removes most of the excess demand at the modelled capacity and the simulated latencies fall with it. These are results for an offline optimum with global knowledge of every layer, computed per layer for the capacity the hardware has; a practical routing algorithm would be a heuristic for the same problem and would be bounded by it. Not shown here: the effect on pre-generation policies (only on-demand was simulated), joint optimisation of mapping and routing, fidelity

effects (the on-demand fidelity does not depend on waiting), and a routing policy that does not know the layer in advance. Whether to pursue the research further is therefore a question about those directions, and the bounds above say how much headroom each could have.

Chapter 9

The torus topology

9.1 What changes

A 4×4 torus joins the opposite edges of the mesh with eight extra links, so every core has four neighbours and the longest path shrinks from six hops to four. Everything else is held as in the mesh experiments: the same ten benchmarks, the same mapper (HQA) with the same seeds, the same ACE parameters, the same link length for every link (a folded layout; the wrap links are not longer), the same scheduler, budgets, policies, physics seeds and controls.

Routing. The routes are the mapper's own: a Dijkstra search for the path with the fewest hops, with the product of link reliabilities as the tie-break (route model `shortest`), exactly as for the mesh traces. The mapping-only path of the study script was checked to reproduce the archived mesh trace line for line before it was used for the torus. ACE itself needed three changes: trace validation knows wrap-around links; the topology file is generated from the mesh file by adding the eight wrap links in the mesh's own link format (every router then has four quantum links, tested by loading it in ACE); and a torus trace is refused on a mesh topology file and the reverse.

One caveat. The mapper is run on the torus graph, so the torus traces differ from the mesh traces in the placement as well as the routes. Differences below are differences of the two systems, not of the topology alone.

9.2 Results

Torus traces need 11% to 37% fewer hops than the mesh traces (CDKM 11%, QFT 23%, QAOA 16%, GHZ 25%, VQE 37%). Every control passed on all 1 368 runs, as for the mesh (Chapter 6).

Workload	Hops	Overload% at C=4	On-demand ms, 2 mem.	On-demand ms, 4 mem.	Dynamic ms, 8 mem.
GHZ	24/18	0.0/0.0	1.17/1.15	1.17/1.15	0.26/0.28
Bernstein–Vazirani	25/21	0.0/0.0	1.22/1.11	1.22/1.11	0.24/0.23
BMW/QUARK cardinality	144/108	0.0/0.0	1.12/1.10	1.12/1.10	0.25/0.25
VQE real amplitudes	104/66	6.1/1.5	1.11/1.14	1.11/1.14	0.25/0.30
CDKM ripple-carry adder	456/406	0.6/0.6	1.24/1.11	1.14/1.09	0.30/0.29
Graph state	76/60	33.3/38.9	2.72/1.86	1.24/1.11	0.95/0.62
Draper QFT adder	1310/984	41.3/25.8	2.16/2.25	1.15/1.13	0.89/0.74
QAOA	3068/2562	71.6/57.9	3.42/2.47	1.27/1.17	1.86/1.10
QFT	2510/1938	65.2/58.0	4.72/2.79	1.59/1.21	2.78/1.16
Random circuit	11688/9574	8.0/3.4	1.27/1.19	1.11/1.11	0.38/0.29

Table 9.1: Mesh/torus. Hops: mean requests per trace. Overload: % of active layers with any core above capacity 4. Latencies are means over nine runs.

On-demand latency under congestion improves: at two memories QFT goes from 4.72 to 2.79 ms, QAOA from 3.42 to 2.47 ms and graph state from 2.72 to 1.86 ms; at four memories QFT from 1.59 to 1.21 ms. Dynamic pre-generation catches on-demand earlier (QFT at eight memories: 2.78 to 1.16 ms). The structural overload at capacity four falls for most workloads (QFT 65.2% to 58.0%, QAOA 71.6% to 57.9%, Draper 41.3% to 25.8%) but not for graph state, where it rises from 33.3% to 38.9% (only six layers are active, so a single layer is 17 percentage points). Draper at two memories is slightly slower on the torus (2.16 to 2.25 ms).

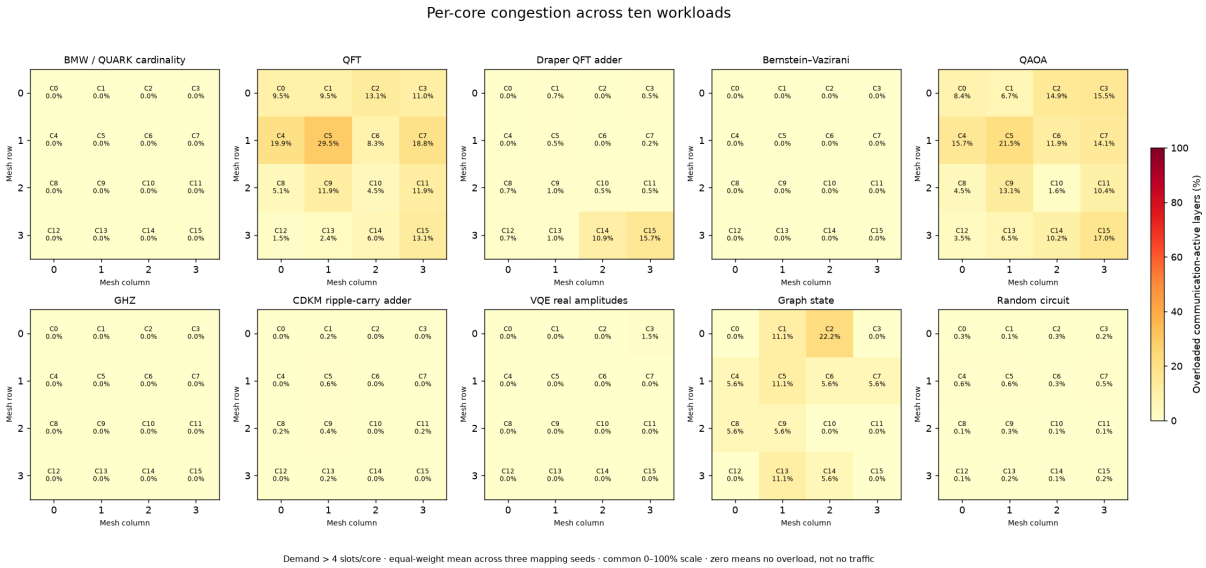


Figure 9.1: Per-core congestion on the torus (percentage of active layers in which a core's demand exceeds 4 slots; mean over three mapping seeds).

Chapter 10

Run times

Wall-clock times on the development machine (8 CPUs). An *invocation* is one process running one benchmark at one budget and policy for three physics seeds. Times from parallel runs include contention from the other jobs; the serial pass was run one invocation at a time and is the clean per-benchmark measurement.

Experiment	Jobs	Runs	Wall clock
Experiment 1, mesh	4	1368	24.5 min
Experiment 1, torus	4	1368	25.2 min
Sensitivity arm	4	120	105.0 s
Serial timing pass	1	270	8.6 min

Table 10.1: Wall-clock time of each experiment.

Benchmark	on-demand, memories					dynamic, memories			
	1	2	4	8	32	2	4	8	32
GHZ	0.4	0.4	0.4	0.4	0.4	0.4	0.4	0.4	0.4
Bernstein-Vazirani	0.4	0.4	0.4	0.4	0.4	0.4	0.4	0.4	0.4
BMW/QUARK cardinality	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
VQE real amplitudes	0.4	0.4	0.4	0.4	0.4	0.4	0.5	0.5	0.5
CDKM ripple-carry adder	0.6	0.6	0.6	0.6	0.6	0.7	0.7	0.7	0.8
Graph state	0.4	0.4	0.4	0.4	0.4	0.4	0.4	0.4	0.4
Draper QFT adder	1.1	1.1	1.1	1.1	1.2	1.2	1.4	1.4	1.6
QAOA	2.1	2.1	2.1	2.2	2.4	2.4	2.8	3.1	3.5
QFT	1.8	1.8	1.7	1.8	2.0	1.9	2.2	2.5	2.9
Random circuit	7.8	7.9	8.0	8.0	8.8	10.6	14.1	14.9	15.3

Table 10.2: Seconds of real time per simulation run, one run at a time (serial pass, mapping seed 42; mean over three physics seeds).

The heatmap regeneration takes 2.7 s and the endpoint-versus-relay split 4.3 s. The new scheduler costs a few percent more real time than the legacy one on the larger traces, because it reads the reservation tables at admission. Real time is unrelated to the simulated latencies the experiments report. The full set of numbers, including parallel per-benchmark sums, is in `docs/reports/run-times.md`.

Chapter 11

Limits and open decisions

- **Physical parameters are undecided.** ACE's coherence time (1 s), memory efficiency (0.95) and 2.5 cm link differ from the group's multi-core paper (1 ms, 0.85). Congestion conclusions are largely structural; pre-generation and fidelity conclusions depend entirely on them (Chapter 6).
- **The scheduler is modelled as a central controller** that reads each core's reservation table instantly. A distributed scheduler would see stale tables.
- **Budgets above four are counterfactual.** They say what more memory would buy, not what the hardware has.
- **One machine, one parameter set, nine runs per cell.** Standard deviations pool mapping and physics seeds.
- **Only on-demand and dynamic** were run in Experiment 1; fixed pre-generation and the autonomous caches were audited for correctness but not swept.
- **Harness constants** (5.3 ms lead, 50 ms window, 5 ms empty-layer gap) dominate makespan on short traces and are identical for both schedulers.
- **No routing policy is simulated.** Chapter 8 bounds what offline routing could achieve structurally and then simulates the optimised routes; it does not claim a general congestion-aware routing algorithm.
- Mechanisms marked "likely" or "probably" in the text were not isolated by an experiment.

Appendix A

Parameters in force

The tables below are generated from the parameter record written by the Experiment 1 driver, which reads the physics from the configuration file and the other constants from the source, so they cannot drift from what ran.

Physical parameter	Value
Memory fidelity	0.95
Memory efficiency	0.95
Memory coherence time	1 s
Memory cut-off ratio (cut-off / coherence)	1 (SeQUeNCe default; not set in the file)
BSM detector efficiencies	0.95, 0.95
Gate / measurement fidelity	0.99 / 0.99
Router-to-BSM fibre	0.025 m, 0.0002 dB/m
Classical delay	distance/c + 20 μs per hop + 100 μs
Configuration file	config/final_config/grid_4x4_ace_3.json

Table A.1: Physical parameters in force (read from the run's parameter record).

Harness constant	Value
Pre-generation lead (request submitted this long before its window)	5.3 ms
Request reservation window	50 ms
Empty-layer gap	5 ms
Retry period after a physical rejection	1 ms
Admission grace after a release or timeout	1 ps
A "wait" means more than	0.001 ms
Compiler reservation window	1000.0 ms
Planner: delta / lookahead / coherence layers	6 / 8 / 10.0
Fidelity threshold of a request	0.01
Simulation stop time	300.0 s
Scheduler	per-qubit, admission event, starvation guard on
Seeds	mapping [42, 43, 44], physics [0, 1, 2]

Table A.2: Harness and scheduler constants (not physical).

A.1 Where the physical values come from, and the open decision

The values were traced to the two neighbouring papers' Table I. ACE's classical delays, detector efficiency, memory and gate fidelities and fibre attenuation match the metro-scale adaptive-continuous protocol paper (arXiv 2502.01964), which also gives a 10 km link, a 2 s coherence time and a memory efficiency of 0.6. ACE's link is 0.025 (read as metres), its coherence time 1 s and its memory efficiency 0.95. The group's multi-core paper (arXiv 2607.14592) states coherence 1 ms and memory efficiency 0.85. The sensitivity arm of Experiment 1 shows that this choice decides every pre-generation and fidelity conclusion; it is the most important open decision, and the question for the supervisor is written in `docs/reports/ace-physical-parameters.md`.

Appendix B

Reproducing the results

All commands are run from the repository root with the project's Python environment. Result folders are dated 2026-10-24.

```
# legacy baseline (must pass after any change to ACE)
python tools/ace_correctness/baseline.py check \
    --golden results/ace_correctness_baseline_2026-10-24/legacy --strict

# tests
python -B -m unittest discover -s test -p 'test_*.py'

# heatmap, Experiment 2
python tools/heatmap/regenerate_heatmap.py --output out/heatmap
python tools/experiment2/make_report.py

# Experiment 1 (mesh), then its analysis
python tools/experiment1/run_experiment1.py --output out --jobs 4
python tools/experiment1/analyze.py --results out/results.csv --output out

# offline routing
python tools/routing_research/run_routing_study.py --output out/routing
python tools/routing_research/simulate_rerouted.py --output out/reroute

# torus: map, then run the experiments
python tools/congestion_study/run_study.py --mapper-dir mapper \
    --topology torus --mapping-only --seeds 42 43 44 --workloads <ten names>
sh tools/torus/run_torus_experiments.sh WORKDIR

# memory cut-off
python tools/cutoff_study/mechanism.py --output out/mechanism.csv

# this report
python tools/latex/make_tables.py
tectonic docs/latex/ace_report/main.tex
```

Every experiment folder contains `run_parameters.json` (all parameters, trace hashes, code commit, machine) and, where timed, `timings.csv`.